

Entrega 3. Ejemplos Java completos, transacciones y gestión de errores

Esta entrega reúne los ejemplos en una estructura Java 8 coherente y reutilizable. El objetivo no es repetir fragmentos aislados, sino mostrar cómo organizar una pequeña capa de acceso a datos SQLJ con contextos, iteradores, transacciones, control de filas afectadas y cierre de recursos.

Los archivos que contienen #sql deben tratarse como fuentes SQLJ, normalmente con extensión .sqlj, y pasar por el traductor SQLJ antes de la compilación Java.

1. Estructura propuesta

```
1  src/  
2    └─ ejemplo/sqlj/  
3        └─ Modelo.java  
4        └─ Iteradores.sqlj  
5        └─ RepositorioSQLJ.sqlj  
6        └─ AplicacionSQLJ.java
```

Responsabilidades:

```
1  Modelo.java  
2    └─ Clases Java utilizadas por la aplicación  
3  
4  Iteradores.sqlj  
5    └─ Declaraciones de iteradores SQLJ  
6  
7  RepositorioSQLJ.sqlj  
8    └─ SELECT  
9    └─ INSERT  
10   └─ UPDATE  
11   └─ DELETE  
12   └─ JOIN  
13   └─ UNION  
14   └─ GROUP BY  
15   └─ Transacciones  
16  
17  AplicacionSQLJ.java
```

18	└─ Obtención de la conexión
19	└─ Creación del contexto
20	└─ Invocación del repositorio
21	└─ Cierre de recursos

SQLJ utiliza `java.sql.SQLException` para comunicar errores SQL. La información relevante puede obtenerse mediante `getSQLState()`, `getErrorCode()` y la cadena de excepciones enlazadas. [\[ibm.com\]](#), [\[ibm.com\]](#)

2. Clases de modelo

2.1. Archivo Modelo.java

```
1  package ejemplo.sqlj;
2
3  import java.math.BigDecimal;
4  import java.sql.Date;
5  import java.util.ArrayList;
6  import java.util.Collections;
7  import java.util.List;
8
9  public final class Modelo {
10
11      private Modelo() {
12      }
13
14      public static final class Cliente {
15
16          private final int idCliente;
17          private final String nombre;
18          private final String email;
19          private final String ciudad;
20
21          public Cliente(
22              int idCliente,
23              String nombre,
24              String email,
25              String ciudad) {
26
27              this.idCliente = idCliente;
```

```
28         this.nombre = nombre;
29         this.email = email;
30         this.ciudad = ciudad;
31     }
32
33     public int getIdCliente() {
34         return idCliente;
35     }
36
37     public String getNombre() {
38         return nombre;
39     }
40
41     public String getEmail() {
42         return email;
43     }
44
45     public String getCiudad() {
46         return ciudad;
47     }
48
49     @Override
50     public String toString() {
51         return "Cliente{" +
52             "idCliente=" + idCliente +
53             ", nombre='" + nombre + '\'' +
54             ", email='" + email + '\'' +
55             ", ciudad='" + ciudad + '\'' +
56             '}';
57     }
58 }
59
60 public static final class Pedido {
61
62     private final int idPedido;
63     private final int idCliente;
64     private final Date fecha;
65     private final BigDecimal importe;
66     private final String estado;
67 }
```

```
68         public Pedido(  
69             int idPedido,  
70             int idCliente,  
71             Date fecha,  
72             BigDecimal importe,  
73             String estado) {  
74  
75             this.idPedido = idPedido;  
76             this.idCliente = idCliente;  
77             this.fecha = fecha;  
78             this.importe = importe;  
79             this.estado = estado;  
80         }  
81  
82         public int getIdPedido() {  
83             return idPedido;  
84         }  
85  
86         public int getIdCliente() {  
87             return idCliente;  
88         }  
89  
90         public Date getFecha() {  
91             return fecha;  
92         }  
93  
94         public BigDecimal getImporte() {  
95             return importe;  
96         }  
97  
98         public String getEstado() {  
99             return estado;  
100        }  
101    }  
102  
103    public static final class LineaPedido {  
104  
105        private final int idProducto;  
106        private final int cantidad;  
107    }
```

```
108         public LineaPedido(int idProducto, int cantidad) {
109             this.idProducto = idProducto;
110             this.cantidad = cantidad;
111         }
112
113         public int getIdProducto() {
114             return idProducto;
115         }
116
117         public int getCantidad() {
118             return cantidad;
119         }
120     }
121
122     public static final class ClientePedido {
123
124         private final int idCliente;
125         private final String nombreCliente;
126         private final Integer idPedido;
127         private final Date fecha;
128         private final BigDecimal importe;
129         private final String estado;
130
131         public ClientePedido(
132             int idCliente,
133             String nombreCliente,
134             Integer idPedido,
135             Date fecha,
136             BigDecimal importe,
137             String estado) {
138
139             this.idCliente = idCliente;
140             this.nombreCliente = nombreCliente;
141             this.idPedido = idPedido;
142             this.fecha = fecha;
143             this.importe = importe;
144             this.estado = estado;
145         }
146
147         public int getIdCliente() {
```

```
148         return idCliente;
149     }
150
151     public String getNombreCliente() {
152         return nombreCliente;
153     }
154
155     public Integer getIdPedido() {
156         return idPedido;
157     }
158
159     public Date getFecha() {
160         return fecha;
161     }
162
163     public BigDecimal getImporte() {
164         return importe;
165     }
166
167     public String getEstado() {
168         return estado;
169     }
170 }
171
172 public static final class DetalleProducto {
173
174     private final int idPedido;
175     private final String cliente;
176     private final int idProducto;
177     private final String producto;
178     private final int cantidad;
179     private final BigDecimal precioUnitario;
180     private final BigDecimal subtotal;
181
182     public DetalleProducto(
183         int idPedido,
184         String cliente,
185         int idProducto,
186         String producto,
187         int cantidad,
```

```
188         BigDecimal precioUnitario,
189         BigDecimal subtotal) {
190
191         this.idPedido = idPedido;
192         this.cliente = cliente;
193         this.idProducto = idProducto;
194         this.producto = producto;
195         this.cantidad = cantidad;
196         this.precioUnitario = precioUnitario;
197         this.subtotal = subtotal;
198     }
199
200     public int getIdPedido() {
201         return idPedido;
202     }
203
204     public String getCliente() {
205         return cliente;
206     }
207
208     public int getIdProducto() {
209         return idProducto;
210     }
211
212     public String getProducto() {
213         return producto;
214     }
215
216     public int getCantidad() {
217         return cantidad;
218     }
219
220     public BigDecimal getPrecioUnitario() {
221         return precioUnitario;
222     }
223
224     public BigDecimal getSubtotal() {
225         return subtotal;
226     }
227 }
```

```
228
229     public static final class ResumenCliente {
230
231         private final int idCliente;
232         private final String nombreCliente;
233         private final long numeroPedidos;
234         private final BigDecimal importeTotal;
235         private final BigDecimal importeMedio;
236
237         public ResumenCliente(
238             int idCliente,
239             String nombreCliente,
240             long numeroPedidos,
241             BigDecimal importeTotal,
242             BigDecimal importeMedio) {
243
244             this.idCliente = idCliente;
245             this.nombreCliente = nombreCliente;
246             this.numeroPedidos = numeroPedidos;
247             this.importeTotal = importeTotal;
248             this.importeMedio = importeMedio;
249         }
250
251         public int getIdCliente() {
252             return idCliente;
253         }
254
255         public String getNombreCliente() {
256             return nombreCliente;
257         }
258
259         public long getNumeroPedidos() {
260             return numeroPedidos;
261         }
262
263         public BigDecimal getImporteTotal() {
264             return importeTotal;
265         }
266
267         public BigDecimal getImporteMedio() {
```



```

268         return importeMedio;
269     }
270 }
271
272 public static final class PedidoCompleto {
273
274     private final Pedido pedido;
275     private final List<LineaPedido> lineas;
276
277     public PedidoCompleto(
278         Pedido pedido,
279         List<LineaPedido> lineas) {
280
281         this.pedido = pedido;
282         this.lineas = Collections.unmodifiableList(
283             new ArrayList<LineaPedido>(lineas)
284         );
285     }
286
287     public Pedido getPedido() {
288         return pedido;
289     }
290
291     public List<LineaPedido> getLineas() {
292         return lineas;
293     }
294 }
295 }

```

3. Declaración de iteradores SQLJ

3.1. Archivo Iteradores.sqlj

```

1  package ejemplo.sqlj;
2
3  import java.math.BigDecimal;
4  import java.sql.Date;
5
6  #sql public iterator ClienteIterator(
7      int idCliente,

```

```

8      String nombre,
9      String email,
10     String ciudad
11 );
12
13 #sql public iterator ClientePedidoIterator(
14     int idCliente,
15     String nombreCliente,
16     Integer idPedido,
17     Date fecha,
18     BigDecimal importe,
19     String estado
20 );
21
22 #sql public iterator DetalleProductoIterator(
23     int idPedido,
24     String cliente,
25     int idProducto,
26     String producto,
27     int cantidad,
28     BigDecimal precioUnitario,
29     BigDecimal subtotal
30 );
31
32 #sql public iterator CiudadIterator(
33     String ciudad
34 );
35
36 #sql public iterator ResumenClienteIterator(
37     int idCliente,
38     String nombreCliente,
39     long numeroPedidos,
40     BigDecimal importeTotal,
41     BigDecimal importeMedio
42 );

```

Los iteradores representan tipos Java generados durante la traducción SQLJ. Los iteradores nombrados permiten acceder a las columnas mediante métodos generados, mientras que los posicionales dependen del orden de las columnas. [\[docs.cloud...oracle.com\]](#), [\[ibm.com\]](#)

4. Repositorio SQLJ completo

4.1. Cabecera del repositorio

Archivo RepositorioSQLJ.sqlj:

```
1  package ejemplo.sqlj;
2
3  import java.math.BigDecimal;
4  import java.sql.Connection;
5  import java.sql.Date;
6  import java.sql.SQLException;
7  import java.util.ArrayList;
8  import java.util.List;
9
10 import sqlj.runtime.ExecutionContext;
11 import sqlj.runtime.ref.DefaultContext;
12
13 import ejemplo.sqlj.Modelo.Cliente;
14 import ejemplo.sqlj.Modelo.ClientePedido;
15 import ejemplo.sqlj.Modelo.DetalleProducto;
16 import ejemplo.sqlj.Modelo.LineaPedido;
17 import ejemplo.sqlj.Modelo.Pedido;
18 import ejemplo.sqlj.Modelo.PedidoCompleto;
19 import ejemplo.sqlj.Modelo.ResumenCliente;
20
21 public class RepositorioSQLJ {
22
23     private final DefaultContext contexto;
24     private final Connection connection;
25
26     public RepositorioSQLJ(
27         Connection connection,
28         DefaultContext contexto) {
29
30         if (connection == null) {
31             throw new IllegalArgumentException(
32                 "La conexión no puede ser null"
33             );
34         }
35
36     }
```

```

36         if (contexto == null) {
37             throw new IllegalArgumentException(
38                 "El contexto SQLJ no puede ser null"
39             );
40         }
41
42         this.connection = connection;
43         this.contexto = contexto;
44     }
45
46     // Los métodos se desarrollan en los apartados siguientes.
47 }

```

El repositorio no abre ni cierra la conexión. La aplicación que crea la conexión es responsable de cerrarla.

Esto evita que una operación individual cierre accidentalmente una conexión compartida.

5. Ejemplo 1: consulta de un cliente por identificador

5.1. Objetivo

Recuperar exactamente un cliente mediante su clave primaria.

```

1  idCliente
2      |
3      ▼
4  SELECT INTO
5      |
6      ▼
7  Objeto Cliente

```

5.2. Método completo

```

1  public Cliente buscarCliente(int idCliente)
2      throws SQLException {
3
4      if (idCliente <= 0) {
5          throw new IllegalArgumentException(
6              "El identificador debe ser positivo"
7          );
8      }

```

```

9
10     String nombre = null;
11     String email = null;
12     String ciudad = null;
13
14     #sql [contexto] {
15         SELECT nombre,
16             email,
17             ciudad
18         INTO :nombre,
19             :email,
20             :ciudad
21         FROM cliente
22         WHERE id_cliente = :idCliente
23     };
24
25     return new Cliente(
26         idCliente,
27         nombre,
28         email,
29         ciudad
30     );
31 }

```

5.3. Comportamiento esperado

- Una fila: devuelve el cliente.
- Ninguna fila: se produce una SQLException o condición equivalente de la implementación.
- Varias filas: se produce una excepción, aunque no debería ocurrir si id_cliente es clave primaria.
- Columna nullable: la variable Java correspondiente puede recibir null.

5.4. Variante segura como búsqueda opcional

SQLJ no dispone de Optional como comportamiento automático. Puede transformarse una condición de “no encontrado” en Optional<Cliente>, pero identificar esa condición exige examinar el SQLState o el código del fabricante.

No debe codificarse una comparación contra el texto del mensaje:

```

1  if (exception.getMessage().contains("not found")) {
2      // Incorrecto y dependiente del idioma.

```

6. Ejemplo 2: inserción de un cliente

6.1. Objetivo

Insertar un cliente y comprobar que se ha añadido exactamente una fila.

```
1  Cliente Java
2      |
3      ▼
4      INSERT
5      |
6      ▼
7  1 fila afectada
```

ExecutionContext permite obtener información de estado de la última operación, incluido el número de filas afectadas mediante `getUpdateCount()`. No debe compartirse simultáneamente el mismo contexto de ejecución entre operaciones concurrentes.

[\[docs.oracle.com\]](#), [\[docs.oracle.com\]](#), [\[ibm.com\]](#)

6.2. Método completo

```
1  public void insertarCliente(Cliente cliente)
2      throws SQLException {
3
4      if (cliente == null) {
5          throw new IllegalArgumentException(
6              "El cliente no puede ser null"
7          );
8      }
9
10     validarCliente(cliente);
11
12     int idCliente = cliente.getIdCliente();
13     String nombre = cliente.getNombre();
14     String email = cliente.getEmail();
15     String ciudad = cliente.getCiudad();
16
17     ExecutionContext ejecucion = new ExecutionContext();
18
19     #sql [contexto, ejecucion] {
```

```
20         INSERT INTO cliente (
21             id_cliente,
22             nombre,
23             email,
24             ciudad
25         )
26         VALUES (
27             :idCliente,
28             :nombre,
29             :email,
30             :ciudad
31         )
32     };
33
34     verificarFilasAfectadas(
35         ejecucion,
36         1,
37         "Insertar cliente"
38     );
39 }
40
41 private void validarCliente(Cliente cliente) {
42
43     if (cliente.getIdCliente() <= 0) {
44         throw new IllegalArgumentException(
45             "El identificador debe ser positivo"
46         );
47     }
48
49     if (cliente.getNombre() == null ||
50         cliente.getNombre().trim().isEmpty()) {
51
52         throw new IllegalArgumentException(
53             "El nombre es obligatorio"
54         );
55     }
56
57     if (cliente.getEmail() == null ||
58         cliente.getEmail().trim().isEmpty()) {
59
```

```

60         throw new IllegalArgumentException(
61             "El correo electrónico es obligatorio"
62         );
63     }
64 }

```

6.3. Consideraciones transaccionales

El método no ejecuta COMMIT.

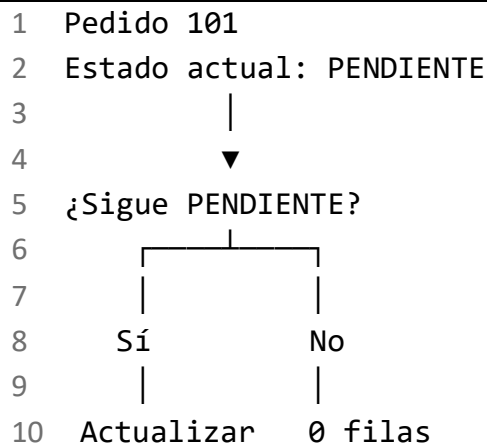
Esto permite que quien lo invoque decida si la inserción:

- Es una operación independiente.
 - Forma parte de una operación mayor.
 - Debe confirmarse junto con otras modificaciones.
 - Debe deshacerse si falla una operación posterior.
-

7. Ejemplo 3: actualización del estado de un pedido

7.1. Objetivo

Actualizar un pedido solamente si se encuentra en el estado esperado.



Este patrón permite detectar cambios concurrentes.

7.2. Método completo

```

1  public void actualizarEstadoPedido(
2      int idPedido,
3      String estadoEsperado,
4      String nuevoEstado) throws SQLException {
5

```



```
6     if (idPedido <= 0) {
7         throw new IllegalArgumentException(
8             "El identificador del pedido debe ser positivo"
9         );
10    }
11
12    if (estadoEsperado == null ||
13        estadoEsperado.trim().isEmpty()) {
14
15        throw new IllegalArgumentException(
16            "El estado esperado es obligatorio"
17        );
18    }
19
20    if (nuevoEstado == null ||
21        nuevoEstado.trim().isEmpty()) {
22
23        throw new IllegalArgumentException(
24            "El nuevo estado es obligatorio"
25        );
26    }
27
28    ExecutionContext ejecucion = new ExecutionContext();
29
30    #sql [contexto, ejecucion] {
31        UPDATE pedido
32        SET estado = :nuevoEstado
33        WHERE id_pedido = :idPedido
34        AND estado = :estadoEsperado
35    };
36
37    verificarFilasAfectadas(
38        ejecucion,
39        1,
40        "Actualizar estado del pedido"
41    );
42 }
```

7.3. Posibles resultados

Una fila afectada

El pedido existía y conservaba el estado esperado.

Cero filas afectadas

Puede significar:

- El pedido no existe.
- Otro proceso modificó el estado.
- El estado proporcionado no coincide.
- La fila fue eliminada.

Más de una fila

Indica un problema grave de integridad, porque id_pedido debería ser único.

8. Ejemplo 4: eliminación de un pedido

8.1. Objetivo

Eliminar las líneas del pedido y después el pedido dentro de la misma transacción.

```
1  LINEA_PEDIDO
2      |
3      ▼
4  Eliminar líneas
5      |
6      ▼
7  PEDIDO
8      |
9      ▼
10 Eliminar cabecera
11      |
12      ▼
13 COMMIT
```

8.2. Método completo

```
1  public void eliminarPedido(int idPedido)
2      throws SQLException {
3
```

```

4      if (idPedido <= 0) {
5          throw new IllegalArgumentException(
6              "El identificador del pedido debe ser positivo"
7          );
8      }
9
10     ejecutarEnTransaccion(new TrabajoTransaccional() {
11         @Override
12         public void ejecutar() throws SQLException {
13
14             ExecutionContext ejecucionLineas =
15                 new ExecutionContext();
16
17             #sql [contexto, ejecucionLineas] {
18                 DELETE FROM linea_pedido
19                 WHERE id_pedido = :idPedido
20             };
21
22             ExecutionContext ejecucionPedido =
23                 new ExecutionContext();
24
25             #sql [contexto, ejecucionPedido] {
26                 DELETE FROM pedido
27                 WHERE id_pedido = :idPedido
28             };
29
30             verificarFilasAfectadas(
31                 ejecucionPedido,
32                 1,
33                 "Eliminar pedido"
34             );
35         }
36     });
37 }

```

8.3. Observaciones

No se exige que DELETE FROM linea_pedido afecte al menos una fila porque podría existir un pedido todavía sin líneas.

Sí se exige que la eliminación de PEDIDO afecte exactamente una fila.

Si la base de datos define:

1 ON DELETE CASCADE

podría eliminarse directamente la cabecera. Esa regla pertenece al esquema y debe verificarse expresamente. No debe asumirse.

9. Ejemplo 5: clientes con pedidos mediante INNER JOIN

9.1. Objetivo

Recuperar únicamente clientes que tienen pedidos.

```
1  CLIENTE
2  |
3  └─ INNER JOIN
4      |
5      ▼
6      PEDIDO
7
8  Solo coincidencias
```

9.2. Método completo

```
1  public List<ClientePedido> buscarClientesConPedidos()
2      throws SQLException {
3
4      List<ClientePedido> resultado =
5          new ArrayList<ClientePedido>();
6
7      ClientePedidoIterator iterador = null;
8
9      try {
10         #sql [contexto] iterador = {
11             SELECT c.id_cliente AS idCliente,
12                    c.nombre AS nombreCliente,
13                    p.id_pedido AS idPedido,
14                    p.fecha AS fecha,
15                    p.importe AS importe,
16                    p.estado AS estado
17             FROM cliente c
18             INNER JOIN pedido p
19                 ON p.id_cliente = c.id_cliente
```

```

20         ORDER BY c.id_cliente, p.id_pedido
21     };
22
23     while (iterador.next()) {
24         resultado.add(
25             new ClientePedido(
26                 iterador.idCliente(),
27                 iterador.nombreCliente(),
28                 iterador.idPedido(),
29                 iterador.fecha(),
30                 iterador.importe(),
31                 iterador.estado()
32             )
33         );
34     }
35
36     return resultado;
37
38 } finally {
39     cerrarIterador(iterador);
40 }
41 }

```

9.3. Resultado conceptual

1	Ana	101	PENDIENTE
2	Ana	102	ENVIADO
3	Luis	104	PENDIENTE
4	Marta	103	ENTREGADO

Los clientes sin pedidos no aparecen.

10. Ejemplo 6: todos los clientes mediante LEFT JOIN

10.1. Objetivo

Recuperar todos los clientes, tengan o no pedidos.

1	CLIENTE
2	
3	└─ LEFT JOIN

4	
5	▼
6	PEDIDO
7	
8	Todos los clientes

10.2. Método completo

```
1  public List<ClientePedido> buscarTodosLosClientesConPedidos()
2      throws SQLException {
3
4      List<ClientePedido> resultado =
5          new ArrayList<ClientePedido>();
6
7      ClientePedidoIterator iterador = null;
8
9      try {
10         #sql [contexto] iterador = {
11             SELECT c.id_cliente AS idCliente,
12                    c.nombre AS nombreCliente,
13                    p.id_pedido AS idPedido,
14                    p.fecha AS fecha,
15                    p.importe AS importe,
16                    p.estado AS estado
17             FROM cliente c
18             LEFT JOIN pedido p
19                 ON p.id_cliente = c.id_cliente
20             ORDER BY c.id_cliente, p.id_pedido
21         };
22
23         while (iterador.next()) {
24             resultado.add(
25                 new ClientePedido(
26                     iterador.idCliente(),
27                     iterador.nombreCliente(),
28                     iterador.idPedido(),
29                     iterador.fecha(),
30                     iterador.importe(),
31                     iterador.estado()
32                 )
33             );
34         }
35     }
36 }
```

```

34         }
35
36         return resultado;
37
38     } finally {
39         cerrarIterador(iterador);
40     }
41 }

```

10.3. Tratamiento de ausencia de pedido

La declaración del iterador utiliza:

```
1 Integer idPedido
```

y no:

```
1 int idPedido
```

Cuando el cliente no tiene pedidos:

```

1 idPedido = null
2 fecha    = null
3 importe  = null
4 estado   = null

```

La aplicación puede interpretarlo así:

```

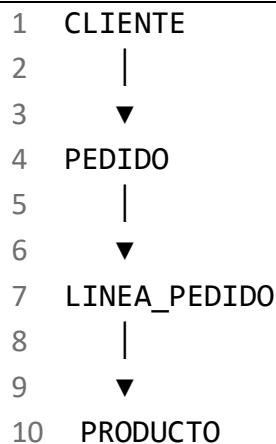
1 for (ClientePedido fila : resultado) {
2     if (fila.getIdPedido() == null) {
3         System.out.println(
4             fila.getNombreCliente() + ": sin pedidos"
5         );
6     } else {
7         System.out.println(
8             fila.getNombreCliente() +
9             ": pedido " +
10            fila.getIdPedido()
11        );
12    }
13 }

```

11. Ejemplo 7: productos y cantidades mediante varias tablas

11.1. Objetivo

Recuperar el detalle de productos de los pedidos combinando cuatro tablas.



11.2. Método completo

```
1  public List<DetalleProducto> buscarDetalleProductos(  
2      int idPedidoBuscado) throws SQLException {  
3  
4      if (idPedidoBuscado <= 0) {  
5          throw new IllegalArgumentException(  
6              "El identificador del pedido debe ser positivo"  
7          );  
8      }  
9  
10     List<DetalleProducto> resultado =  
11         new ArrayList<DetalleProducto>();  
12  
13     DetalleProductoIterator iterador = null;  
14  
15     try {  
16         #sql [contexto] iterador = {  
17             SELECT p.id_pedido AS idPedido,  
18                 c.nombre AS cliente,  
19                 pr.id_producto AS idProducto,  
20                 pr.nombre AS producto,  
21                 lp.cantidad AS cantidad,
```



```

22         pr.precio AS precioUnitario,
23         lp.cantidad * pr.precio AS subtotal
24     FROM cliente c
25     INNER JOIN pedido p
26         ON p.id_cliente = c.id_cliente
27     INNER JOIN linea_pedido lp
28         ON lp.id_pedido = p.id_pedido
29     INNER JOIN producto pr
30         ON pr.id_producto = lp.id_producto
31     WHERE p.id_pedido = :idPedidoBuscado
32     ORDER BY pr.nombre
33 };
34
35     while (iterador.next()) {
36         resultado.add(
37             new DetalleProducto(
38                 iterador.idPedido(),
39                 iterador.cliente(),
40                 iterador.idProducto(),
41                 iterador.producto(),
42                 iterador.cantidad(),
43                 iterador.precioUnitario(),
44                 iterador.subtotal()
45             )
46         );
47     }
48
49     return resultado;
50
51 } finally {
52     cerrarIterador(iterador);
53 }
54 }

```

11.3. Resultado conceptual

```

1  Pedido 101
2  |— Ratón
3  |   |— Cantidad: 1
4  |   |— Precio: 25.00
5  |   └— Subtotal: 25.00

```

```
6 |  
7 |└─ Teclado  
8 |   └─ Cantidad: 2  
9 |   └─ Precio: 40.00  
10 |   └─ Subtotal: 80.00
```

Para importes se utiliza BigDecimal, no double.

12. Ejemplo 8: unión de resultados mediante UNION

12.1. Objetivo

Combinar ciudades procedentes de clientes y ciudades proporcionadas como parámetros, eliminando duplicados.

El número de columnas y los tipos deben ser compatibles en ambos lados del UNION.

12.2. Método completo

```
1  public List<String> combinarCiudades(  
2      String ciudadAdicional1,  
3      String ciudadAdicional2) throws SQLException {  
4  
5      if (ciudadAdicional1 == null ||  
6          ciudadAdicional2 == null) {  
7  
8          throw new IllegalArgumentException(  
9              "Las ciudades adicionales son obligatorias"  
10         );  
11     }  
12  
13     List<String> resultado = new ArrayList<String>();  
14     CiudadIterator iterador = null;  
15  
16     try {  
17         #sql [contexto] iterador = {  
18             SELECT ciudad AS ciudad  
19             FROM cliente  
20  
21             UNION  
22  
23             SELECT :ciudadAdicional1 AS ciudad
```

```

24         FROM cliente
25         WHERE id_cliente = (
26             SELECT MIN(id_cliente)
27             FROM cliente
28         )
29
30         UNION
31
32         SELECT :ciudadAdicional2 AS ciudad
33         FROM cliente
34         WHERE id_cliente = (
35             SELECT MIN(id_cliente)
36             FROM cliente
37         )
38
39         ORDER BY ciudad
40     };
41
42     while (iterador.next()) {
43         resultado.add(iterador.ciudad());
44     }
45
46     return resultado;
47
48 } finally {
49     cerrarIterador(iterador);
50 }
51 }

```

12.3. Observación de portabilidad

La segunda y tercera consultas utilizan CLIENTE para producir una fila sin depender de una tabla auxiliar concreta.

Algunos fabricantes permiten:

```
1 SELECT 'Bilbao'
```

sin cláusula FROM. Otros productos o versiones pueden requerir una tabla especial, como DUAL en determinados entornos Oracle.

Para mantener el ejemplo independiente del fabricante se utiliza una tabla del modelo.

13. Ejemplo 9: informe agrupado por cliente

13.1. Objetivo

Obtener:

- Número de pedidos.
- Importe total.
- Importe medio.
- Clientes sin pedidos incluidos.

```
1  CLIENTE
2  |
3  ▼
4  LEFT JOIN PEDIDO
5  |
6  ▼
7  GROUP BY cliente
8  |
9  ▼
10 COUNT, SUM y AVG
```

13.2. Método completo

```
1  public List<ResumenCliente> obtenerInformePorCliente()
2      throws SQLException {
3
4      List<ResumenCliente> resultado =
5          new ArrayList<ResumenCliente>();
6
7      ResumenClienteIterator iterador = null;
8
9      try {
10         #sql [contexto] iterador = {
11             SELECT c.id_cliente AS idCliente,
12                    c.nombre AS nombreCliente,
13                    COUNT(p.id_pedido) AS numeroPedidos,
14                    COALESCE(SUM(p.importe), 0) AS
importeTotal,
15                    COALESCE(AVG(p.importe), 0) AS importeMedio
16             FROM cliente c
17             LEFT JOIN pedido p
18                 ON p.id_cliente = c.id_cliente
```

```

19         GROUP BY c.id_cliente, c.nombre
20         ORDER BY c.id_cliente
21     };
22
23     while (iterador.next()) {
24         resultado.add(
25             new ResumenCliente(
26                 iterador.idCliente(),
27                 iterador.nombreCliente(),
28                 iterador.numeroPedidos(),
29                 iterador.importeTotal(),
30                 iterador.importeMedio()
31             )
32         );
33     }
34
35     return resultado;
36
37 } finally {
38     cerrarIterador(iterador);
39 }
40 }

```

13.3. Por qué se utiliza COUNT(p.id_pedido)

Con LEFT JOIN, un cliente sin pedidos genera una fila lógica con columnas de PEDIDO a NULL.

```
1 COUNT(*)
```

podría contar esa fila.

En cambio:

```
1 COUNT(p.id_pedido)
```

cuenta únicamente los identificadores de pedido no nulos.

Resultado correcto:

```
1 Pablo | 0 pedidos
```

13.4. Implicación de COALESCE

```
1 COALESCE(SUM(p.importe), 0)
```

transforma ausencia de importes en cero.

Debe utilizarse solamente si funcionalmente se acepta que:

1 sin pedidos = importe total cero

14. Ejemplo 10: transacción para crear un pedido y sus líneas

14.1. Objetivo

Crear de forma atómica:

1. La cabecera del pedido.
2. Todas las líneas.
3. El importe total.
4. La actualización del importe.
5. El COMMIT.

Si cualquier operación falla, se ejecuta ROLLBACK.

1	Inicio
2	
3	▼
4	Validar pedido y líneas
5	
6	▼
7	INSERT PEDIDO
8	
9	▼
10	Comprobar productos
11	
12	▼
13	INSERT LINEAS
14	
15	▼
16	Calcular total
17	
18	▼
19	UPDATE PEDIDO
20	
21	▼
22	COMMIT

Ante cualquier error:

```
1  Error
2  |
3  ▼
4  ROLLBACK
5  |
6  ▼
7  No se conserva ningún cambio
```

14.2. Método completo

```
1  public void crearPedidoCompleto(
2      final PedidoCompleto pedidoCompleto)
3      throws SQLException {
4
5      if (pedidoCompleto == null) {
6          throw new IllegalArgumentException(
7              "El pedido completo no puede ser null"
8          );
9      }
10
11     validarPedidoCompleto(pedidoCompleto);
12
13     ejecutarEnTransaccion(new TrabajoTransaccional() {
14         @Override
15         public void ejecutar() throws SQLException {
16
17             Pedido pedido = pedidoCompleto.getPedido();
18
19             int idPedido = pedido.getIdPedido();
20             int idCliente = pedido.getIdCliente();
21             Date fecha = pedido.getFecha();
22             String estado = pedido.getEstado();
23
24             BigDecimal importeInicial = BigDecimal.ZERO;
25
26             ExecutionContext insercionPedido =
27                 new ExecutionContext();
28
29             #sql [contexto, insercionPedido] {
```

```
30         INSERT INTO pedido (
31             id_pedido,
32             id_cliente,
33             fecha,
34             importe,
35             estado
36         )
37         VALUES (
38             :idPedido,
39             :idCliente,
40             :fecha,
41             :importeInicial,
42             :estado
43         )
44     };
45
46     verificarFilasAfectadas(
47         insercionPedido,
48         1,
49         "Insertar cabecera del pedido"
50     );
51
52     BigDecimal importeTotal = BigDecimal.ZERO;
53
54     for (LineaPedido linea :
55         pedidoCompleto.getLineas()) {
56
57         int idProducto = linea.getIdProducto();
58         int cantidad = linea.getCantidad();
59
60         BigDecimal precio = buscarPrecioProducto(
61             idProducto
62         );
63
64         importeTotal = importeTotal.add(
65             precio.multiply(
66                 BigDecimal.valueOf(cantidad)
67             )
68         );
69     }
```



```

70         ExecutionContext insercionLinea =
71             new ExecutionContext();
72
73         #sql [contexto, insercionLinea] {
74             INSERT INTO linea_pedido (
75                 id_pedido,
76                 id_producto,
77                 cantidad
78             )
79             VALUES (
80                 :idPedido,
81                 :idProducto,
82                 :cantidad
83             )
84         };
85
86         verificarFilasAfectadas(
87             insercionLinea,
88             1,
89             "Insertar línea de pedido"
90         );
91     }
92
93     ExecutionContext actualizacionImporte =
94         new ExecutionContext();
95
96     #sql [contexto, actualizacionImporte] {
97         UPDATE pedido
98         SET importe = :importeTotal
99         WHERE id_pedido = :idPedido
100     };
101
102     verificarFilasAfectadas(
103         actualizacionImporte,
104         1,
105         "Actualizar importe del pedido"
106     );
107 }
108 });
109 }

```

14.3. Consulta del precio y comprobación del producto

```
1  private BigDecimal buscarPrecioProducto(int idProducto)
2      throws SQLException {
3
4      BigDecimal precio = null;
5
6      #sql [contexto] {
7          SELECT precio
8          INTO :precio
9          FROM producto
10         WHERE id_producto = :idProducto
11     };
12
13     if (precio == null) {
14         throw new SQLException(
15             "El producto no tiene un precio informado",
16             "22004"
17         );
18     }
19
20     if (precio.signum() < 0) {
21         throw new SQLException(
22             "El producto tiene un precio negativo",
23             "22003"
24         );
25     }
26
27     return precio;
28 }
```

Si el producto no existe, el SELECT INTO falla antes de insertar la línea. La transacción completa se revierte.

14.4. Validación previa

```
1  private void validarPedidoCompleto(
2      PedidoCompleto pedidoCompleto) {
3
4      Pedido pedido = pedidoCompleto.getPedido();
```

```
5
6     if (pedido == null) {
7         throw new IllegalArgumentException(
8             "La cabecera del pedido es obligatoria"
9         );
10    }
11
12    if (pedido.getIdPedido() <= 0) {
13        throw new IllegalArgumentException(
14            "El identificador del pedido debe ser positivo"
15        );
16    }
17
18    if (pedido.getIdCliente() <= 0) {
19        throw new IllegalArgumentException(
20            "El identificador del cliente debe ser positivo"
21        );
22    }
23
24    if (pedido.getFecha() == null) {
25        throw new IllegalArgumentException(
26            "La fecha del pedido es obligatoria"
27        );
28    }
29
30    if (pedido.getEstado() == null ||
31        pedido.getEstado().trim().isEmpty()) {
32
33        throw new IllegalArgumentException(
34            "El estado del pedido es obligatorio"
35        );
36    }
37
38    if (pedidoCompleto.getLineas().isEmpty()) {
39        throw new IllegalArgumentException(
40            "El pedido debe contener al menos una línea"
41        );
42    }
43
44    for (LineaPedido linea :
```

```

45         pedidoCompleto.getLineas()) {
46
47         if (linea == null) {
48             throw new IllegalArgumentException(
49                 "Una línea de pedido no puede ser null"
50             );
51         }
52
53         if (linea.getIdProducto() <= 0) {
54             throw new IllegalArgumentException(
55                 "El producto debe tener un identificador
56                 válido"
57             );
58         }
59
60         if (linea.getCantidad() <= 0) {
61             throw new IllegalArgumentException(
62                 "La cantidad debe ser mayor que cero"
63             );
64         }
65     }

```

La validación Java no sustituye las restricciones de la base de datos.

Deben seguir existiendo restricciones como:

```

1  PK PEDIDO
2  PK PRODUCTO
3  PK LINEA_PEDIDO
4  FK PEDIDO → CLIENTE
5  FK LINEA_PEDIDO → PEDIDO
6  FK LINEA_PEDIDO → PRODUCTO
7  CHECK cantidad > 0
8  CHECK precio >= 0

```

15. Infraestructura común de transacciones

15.1. Interfaz de trabajo transaccional

Java 8 permite expresar esta interfaz como una interfaz funcional:

```
1 @FunctionalInterface
2 private interface TrabajoTransaccional {
3
4     void ejecutar() throws SQLException;
5 }
```

15.2. Ejecutor transaccional completo

```
1 private void ejecutarEnTransaccion(
2     TrabajoTransaccional trabajo)
3     throws SQLException {
4
5     boolean autoCommitOriginal =
6         connection.setAutoCommit();
7
8     SQLException errorPrincipal = null;
9
10    try {
11        if (autoCommitOriginal) {
12            connection.setAutoCommit(false);
13        }
14
15        trabajo.ejecutar();
16
17        connection.commit();
18
19    } catch (SQLException exception) {
20        errorPrincipal = exception;
21
22        try {
23            connection.rollback();
24        } catch (SQLException rollbackException) {
25            exception.addSuppressed(rollbackException);
26        }
27
28        throw exception;
29
30    } catch (RuntimeException exception) {
31        try {
32            connection.rollback();
```

```

33         } catch (SQLException rollbackException) {
34             exception.addSuppressed(rollbackException);
35         }
36
37         throw exception;
38
39     } finally {
40         if (autoCommitOriginal) {
41             try {
42                 connection.setAutoCommit(true);
43             } catch (SQLException restoreException) {
44                 if (errorPrincipal != null) {
45                     errorPrincipal.addSuppressed(
46                         restoreException
47                     );
48                 } else {
49                     throw restoreException;
50                 }
51             }
52         }
53     }
54 }

```

15.3. Por qué se usa connection.commit()

También podría escribirse:

```

1  #sql [contexto] {
2      COMMIT
3  };

```

y:

```

1  #sql [contexto] {
2      ROLLBACK
3  };

```

En el ejemplo se utiliza la conexión JDBC porque:

- La conexión ya controla autoCommit.
- El bloque de errores puede operar directamente sobre ella.
- Evita mezclar dos estilos de control transaccional.
- Permite conservar con claridad la excepción original.

Esto no convierte las operaciones SQLJ en operaciones JDBC. Únicamente centraliza el control de la transacción en la conexión subyacente.

La conexión asociada al contexto SQLJ y la conexión sobre la que se ejecuta `commit()` o `rollback()` deben ser la misma.

15.4. Limitación importante del método

El método restaura `autoCommit` solo cuando inicialmente estaba activado.

Si recibe una conexión que ya participa en una transacción externa:

```
1 autoCommitOriginal = false
```

el método actual ejecutaría igualmente `commit()`, apropiándose de una transacción que quizá pertenezca al llamador.

En sistemas con transacciones externas conviene separar dos métodos:

```
1 crearPedidoCompleto()  
2     └─ Controla su propia transacción  
3  
4 insertarPedidoEnTransaccionExistente()  
5     └─ No ejecuta COMMIT ni ROLLBACK
```

Una alternativa más rigurosa es impedir que el método transaccional reciba conexiones con `autoCommit` desactivado:

```
1 if (!connection.getAutoCommit()) {  
2     throw new SQLException(  
3         "La conexión ya participa en una transacción externa"  
4     );  
5 }
```

La política correcta depende de la arquitectura de la aplicación.

16. Utilidades de filas afectadas y cierre

16.1. Comprobación de filas afectadas

```
1 private void verificarFilasAfectadas(  
2     ExecutionContext ejecucion,  
3     int esperado,  
4     String operacion) throws SQLException {  
5
```

```

6      int actual = ejecucion.getUpdateCount();
7
8      if (actual != esperado) {
9          throw new SQLException(
10              operacion +
11              ": se esperaban " +
12              esperado +
13              " filas afectadas, pero se obtuvieron " +
14              actual
15          );
16      }
17  }

```

IBM documenta explícitamente el patrón de asociar un ExecutionContext a una cláusula SQL y consultar después getUpdateCount(). [\[ibm.com\]](http://ibm.com)

16.2. Cierre de iteradores

Los iteradores generados no comparten necesariamente una interfaz de cierre suficientemente cómoda para un único método sobrecargado. Por claridad, pueden declararse sobrecargas:

```

1  private void cerrarIterador(
2      ClientePedidoIterator iterador)
3      throws SQLException {
4
5      if (iterador != null) {
6          iterador.close();
7      }
8  }
9
10 private void cerrarIterador(
11     DetalleProductoIterator iterador)
12     throws SQLException {
13
14     if (iterador != null) {
15         iterador.close();
16     }
17 }
18
19 private void cerrarIterador(

```



```

20         CiudadIterator iterador)
21         throws SQLException {
22
23         if (iterador != null) {
24             iterador.close();
25         }
26     }
27
28     private void cerrarIterador(
29         ResumenClienteIterator iterador)
30         throws SQLException {
31
32         if (iterador != null) {
33             iterador.close();
34         }
35     }

```

Si el procesamiento lanza una excepción y el cierre también falla, un finally sencillo puede ocultar el error original. Para evitarlo puede usarse un patrón más completo:

```

1     private SQLException cerrarSinOcultar(
2         ClientePedidoIterator iterador,
3         SQLException errorPrincipal) {
4
5         if (iterador == null) {
6             return errorPrincipal;
7         }
8
9         try {
10             iterador.close();
11         } catch (SQLException closeException) {
12             if (errorPrincipal == null) {
13                 return closeException;
14             }
15
16             errorPrincipal.addSuppressed(closeException);
17         }
18
19         return errorPrincipal;
20     }

```

Para facilitar la lectura, los ejemplos anteriores utilizan la versión directa.

17. Aplicación completa

17.1. Archivo AplicacionSQLJ.java

El URL JDBC, el nombre del controlador y las propiedades de conexión dependen del fabricante.

```
1  package ejemplo.sqlj;
2
3  import java.math.BigDecimal;
4  import java.sql.Connection;
5  import java.sql.Date;
6  import java.sql.DriverManager;
7  import java.sql.SQLException;
8  import java.util.Arrays;
9  import java.util.List;
10 import java.util.Properties;
11
12 import sqlj.runtime.ref.DefaultContext;
13
14 import ejemplo.sqlj.Modelo.Cliente;
15 import ejemplo.sqlj.Modelo.ClientePedido;
16 import ejemplo.sqlj.Modelo.DetalleProducto;
17 import ejemplo.sqlj.Modelo.LineaPedido;
18 import ejemplo.sqlj.Modelo.Pedido;
19 import ejemplo.sqlj.Modelo.PedidoCompleto;
20 import ejemplo.sqlj.Modelo.ResumenCliente;
21
22 public class AplicacionSQLJ {
23
24     public static void main(String[] args) {
25
26         Connection connection = null;
27         DefaultContext contexto = null;
28
29         try {
30             Properties propiedades = new Properties();
31
32             propiedades.setProperty(
```

```
33         "user",
34         obtenerVariableEntorno("DB_USER")
35     );
36
37     propiedades.setProperty(
38         "password",
39         obtenerVariableEntorno("DB_PASSWORD")
40     );
41
42     String url = obtenerVariableEntorno("DB_URL");
43
44     connection = DriverManager.getConnection(
45         url,
46         propiedades
47     );
48
49     contexto = new DefaultContext(connection);
50
51     RepositorioSQLJ repositorio =
52         new RepositorioSQLJ(
53             connection,
54             contexto
55         );
56
57     ejecutarEjemplos(repositorio);
58
59 } catch (SQLException exception) {
60     registrarErrorSQL(exception);
61
62 } catch (RuntimeException exception) {
63     System.err.println(
64         "Error de aplicación: " +
65         exception.getMessage()
66     );
67
68 } finally {
69     cerrarRecursos(contexto, connection);
70 }
71 }
72
```

```
73     private static void ejecutarEjemplos(  
74         RepositorioSQLJ repositorio)  
75         throws SQLException {  
76  
77         Cliente cliente =  
78             repositorio.buscarCliente(1);  
79  
80         System.out.println(cliente);  
81  
82         List<ClientePedido> clientesConPedidos =  
83             repositorio.buscarClientesConPedidos();  
84  
85         for (ClientePedido fila : clientesConPedidos) {  
86             System.out.println(  
87                 fila.getNombreCliente() +  
88                 " | " +  
89                 fila.getIdPedido()  
90             );  
91         }  
92  
93         List<DetalleProducto> productos =  
94             repositorio.buscarDetalleProductos(101);  
95  
96         for (DetalleProducto producto : productos) {  
97             System.out.println(  
98                 producto.getProducto() +  
99                 " | " +  
100                 producto.getCantidad() +  
101                 " | " +  
102                 producto.getSubtotal()  
103             );  
104         }  
105  
106         List<ResumenCliente> resumen =  
107             repositorio.obtenerInformePorCliente();  
108  
109         for (ResumenCliente fila : resumen) {  
110             System.out.println(  
111                 fila.getNombreCliente() +  
112                 " | pedidos: " +
```

```
113         fila.getNumeroPedidos() +
114         " | total: " +
115         fila.getImporteTotal()
116     );
117 }
118
119 Pedido pedido = new Pedido(
120     200,
121     1,
122     Date.valueOf("2026-09-08"),
123     BigDecimal.ZERO,
124     "PENDIENTE"
125 );
126
127 List<LineaPedido> lineas = Arrays.asList(
128     new LineaPedido(10, 2),
129     new LineaPedido(11, 1)
130 );
131
132 repositorio.crearPedidoCompleto(
133     new PedidoCompleto(
134         pedido,
135         lineas
136     )
137 );
138 }
139
140 private static String obtenerVariableEntorno(
141     String nombre) {
142
143     String valor = System.getenv(nombre);
144
145     if (valor == null || valor.trim().isEmpty()) {
146         throw new IllegalStateException(
147             "No está definida la variable de entorno: " +
148             nombre
149         );
150     }
151
152     return valor;
```

```
153     }
154
155     private static void registrarErrorSQL(
156         SQLException exception) {
157
158         SQLException actual = exception;
159
160         while (actual != null) {
161             System.err.println(
162                 "Error SQL" +
163                 " | SQLState=" + actual.getSQLState() +
164                 " | código=" + actual.getErrorCode() +
165                 " | mensaje=" + actual.getMessage()
166             );
167
168             for (Throwable suppressed :
169                 actual.getSuppressed()) {
170
171                 System.err.println(
172                     "Error suprimido: " +
173                     suppressed.getMessage()
174                 );
175             }
176
177             actual = actual.getNextException();
178         }
179     }
180
181     private static void cerrarRecursos(
182         DefaultContext contexto,
183         Connection connection) {
184
185         SQLException errorCierre = null;
186
187         if (contexto != null) {
188             try {
189                 contexto.close();
190             } catch (SQLException exception) {
191                 errorCierre = exception;
192             }
193         }
```

```

193     }
194
195     if (connection != null) {
196         try {
197             if (!connection.isClosed()) {
198                 connection.close();
199             }
200         } catch (SQLException exception) {
201             if (errorCierre == null) {
202                 errorCierre = exception;
203             } else {
204                 errorCierre.addSuppressed(exception);
205             }
206         }
207     }
208
209     if (errorCierre != null) {
210         registrarErrorSQL(errorCierre);
211     }
212 }
213 }

```

18. Gestión de errores SQL

18.1. Información disponible

Una SQLException proporciona:

```

1  exception.getMessage();
2  exception.getSQLState();
3  exception.getErrorCode();
4  exception.getNextException();
5  exception.getCause();
6  exception.getSuppressed();

```

Interpretación:

```

1  getMessage()
2      └─ Descripción textual
3
4  getSQLState()
5      └─ Categoría SQL estandarizada o definida por proveedor

```

```
6
7  getErrorCode()
8      └─ Código numérico del fabricante
9
10 getNextException()
11     └─ Siguiete error de la cadena SQL
12
13 getSuppressed()
14     └─ Errores secundarios, por ejemplo un rollback fallido
```

IBM recomienda obtener `SQLState` y código del fabricante desde la `SQLException`; también puede ofrecer información adicional específica del controlador. [\[ibm.com\]](#), [\[ibm.com\]](#)

18.2. Clasificación básica por `SQLState`

Los dos primeros caracteres de `SQLState` suelen identificar una clase de error:

```
1  private static String clasificar(SQLException exception) {
2
3      String sqlState = exception.getSQLState();
4
5      if (sqlState == null || sqlState.length() < 2) {
6          return "ERROR_SQL_NO_CLASIFICADO";
7      }
8
9      String clase = sqlState.substring(0, 2);
10
11     if ("08".equals(clase)) {
12         return "ERROR_CONEXION";
13     }
14
15     if ("22".equals(clase)) {
16         return "ERROR_DATOS";
17     }
18
19     if ("23".equals(clase)) {
20         return "VIOLACION_INTEGRIDAD";
21     }
22
23     if ("40".equals(clase)) {
24         return "ROLLBACK_TRANSACCION";
25     }
26 }
```



```

25     }
26
27     return "ERROR_SQL";
28 }

```

La interpretación concreta debe contrastarse con la documentación del fabricante. No todos los productos utilizan exactamente los mismos códigos detallados.

19. Errores habituales y tratamiento recomendado

Error	Síntoma	Detección	Tratamiento	¿Rollback?
Ninguna fila en SELECT INTO	La consulta no asigna resultado	SQLException, SQLState o código del fabricante	Convertir en “no encontrado” si es esperado	No, salvo que forme parte de una transacción modificadora
Varias filas en SELECT INTO	La consulta viola la cardinalidad esperada	SQLException	Corregir el filtro o utilizar iterador	Normalmente sí si hay modificaciones pendientes
Clave primaria duplicada	Falla un INSERT	Clase SQLState de integridad	Informar de conflicto o generar correctamente el ID	Sí
Clave ajena inválida	Falla INSERT, UPDATE o DELETE	SQLState y código del fabricante	Validar relación y mantener restricciones	Sí
NULL inesperado	Error de conversión o valor null	Excepción o comprobación Java	Usar envoltorios y validar semántica	Depende de la operación
Tipo incompatible	Error de conversión	Traducción SQLJ o ejecución	Corregir mapeo Java-SQL	Sí si hay modificaciones
Error de conexión	Operación no ejecutada o conexión perdida	SQLState clase 08 habitualmente	Invaldar conexión y reintentar solo si es seguro	Estado incierto
Bloqueo o timeout	Espera y posterior excepción	SQLState/código del proveedor	Reintento limitado y controlado	Sí
Error de COMMIT	No puede confirmarse la transacción	Excepción en commit()	Considerar resultado incierto	No asumir éxito

Iterador no cerrado	Recursos retenidos	Monitorización o agotamiento de cursores	Cerrar en finally	No necesariamente
SQL no portable	Traducción o ejecución incorrecta	Error del traductor o base de datos	Aislar sintaxis específica	Depende

20. Tratamiento detallado de situaciones críticas

20.1. Violación de clave primaria

```

1  INSERT PEDIDO 101
2      |
3      ▼
4  PEDIDO 101 ya existe
5      |
6      ▼
7  SQLException
8      |
9      ▼
10 ROLLBACK

```

Tratamiento:

```

1  catch (SQLException exception) {
2      if (esViolacionIntegridad(exception)) {
3          // Traducir a una excepción funcional.
4      }
5
6      throw exception;
7  }

```

No debe identificarse únicamente por el mensaje:

```

1  exception.getMessage().contains("duplicate")

```

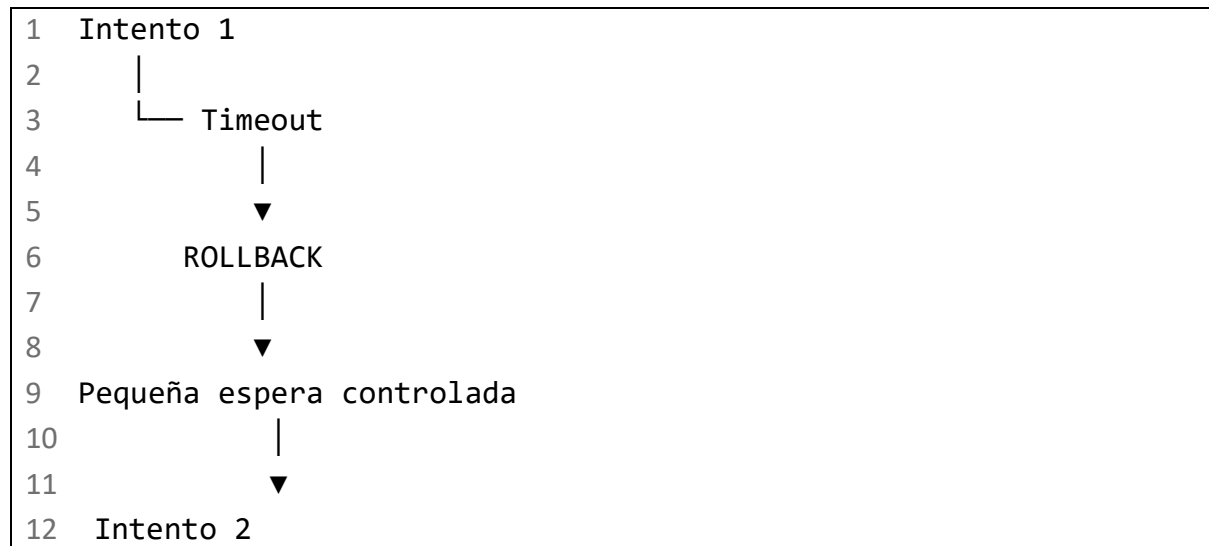
El mensaje puede variar por:

- Fabricante.
- Versión.
- Idioma.
- Configuración.
- Controlador.

20.2. Bloqueo o timeout

Un reintento solamente es seguro cuando la operación completa es idempotente o dispone de controles contra duplicados.

Patrón conceptual:

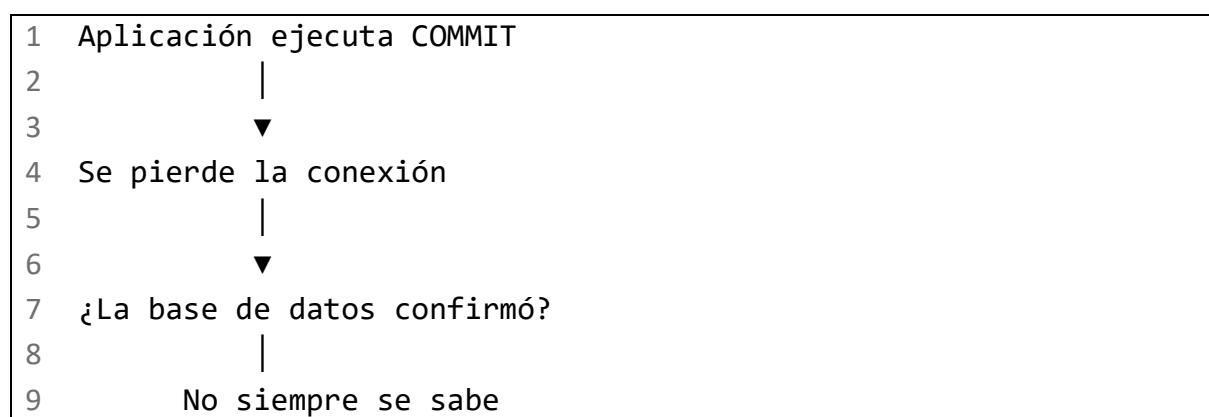


No debe aplicarse un bucle infinito.

En operaciones como crear un pedido, antes de reintentar debe conocerse si la primera transacción quedó confirmada, revertida o en estado incierto.

20.3. Error durante COMMIT

Este es uno de los casos más delicados:



La aplicación no debe ejecutar automáticamente otra inserción como si nada hubiera ocurrido. Podría duplicar la operación.

Medidas:

1. Utilizar identificadores de operación únicos.
2. Diseñar operaciones idempotentes.

3. Consultar el estado antes de repetir.
 4. Invalidar la conexión.
 5. Registrar un identificador de correlación.
 6. No registrar datos sensibles.
-

21. Gestión correcta de recursos

21.1. Orden recomendado

```
1  Abrir conexión
2      |
3      ▼
4  Crear contexto SQLJ
5      |
6      ▼
7  Ejecutar operación
8      |
9      ▼
10 Cerrar iteradores
11     |
12     ▼
13 Cerrar contexto
14     |
15     ▼
16 Cerrar conexión
```

21.2. Propiedad de los recursos

Debe decidirse claramente quién crea y quién cierra cada recurso.

```
1  AplicacionSQLJ
2  |— crea Connection
3  |— crea DefaultContext
4  |— crea RepositorioSQLJ
5  |— invoca operaciones
6  |— cierra DefaultContext
7  |— cierra Connection
8
9  RepositorioSQLJ
10 |— recibe Connection
11 |— recibe DefaultContext
```

12		—	crea iteradores
13		—	cierra sus iteradores

Un método del repositorio no debería cerrar una conexión que recibió del llamador.

22. Consideraciones específicas de implementación

Los ejemplos utilizan elementos ampliamente reconocibles de SQLJ:

- #sql.
- Variables host con :.
- DefaultContext.
- ExecutionContext.
- Iteradores nombrados.
- SELECT INTO.
- Cláusulas SQL ejecutables.

Sin embargo, deben verificarse para el entorno real:

1. Constructores disponibles de DefaultContext.
2. Comportamiento de DefaultContext.close().
3. Correspondencia exacta de alias e iteradores.
4. Reglas de traducción online y offline.
5. Artefactos de perfil generados.
6. Proceso de personalización o bind.
7. Soporte del dialecto SQL.
8. Correspondencia de tipos DATE, TIMESTAMP y DECIMAL.
9. Comportamiento de iteradores después de COMMIT o ROLLBACK.
10. Integración con el pool de conexiones.

Oracle documenta los contextos de conexión, los contextos de ejecución, los iteradores y el efecto de las transacciones sobre resultados e iteradores como aspectos diferenciados del runtime SQLJ. [\[docs.oracle.com\]](https://docs.oracle.com), [\[docs.cloud...oracle.com\]](https://docs.cloud.oracle.com)

23. Resumen visual de la entrega

1	APLICACIÓN JAVA		
2			
3		—	Connection
4			— autoCommit

5			— commit()
6			— rollback()
7			— close()
8			
9			— DefaultContext
10			— Conecta SQLJ con la conexión
11			
12			— RepositorioSQLJ
13			
14			— SELECT INTO
15			— buscarCliente()
16			
17			— INSERT
18			— insertarCliente()
19			
20			— UPDATE
21			— actualizarEstadoPedido()
22			
23			— DELETE
24			— eliminarPedido()
25			
26			— INNER JOIN
27			— buscarClientesConPedidos()
28			
29			— LEFT JOIN
30			— buscarTodosLosClientesConPedidos()
31			
32			— JOIN múltiple
33			— buscarDetalleProductos()
34			
35			— UNION
36			— combinarCiudades()
37			
38			— GROUP BY
39			— obtenerInformePorCliente()
40			
41			— Transacción
42			— crearPedidoCompleto()
43			
44			— ExecutionContext

```
45 |   └─ Verifica filas afectadas
46 |
47 | └─ Iteradores
48 |   └─ next()
49 |   └─ Métodos de columna
50 |   └─ close()
51 |
52 | └─ SQLException
53 |   └─ SQLState
54 |   └─ errorCode
55 |   └─ nextException
56 |   └─ suppressed
```

24. Lista de comprobación

Después de esta tercera entrega, el alumno debería poder:

- Estructurar una aplicación SQLJ en varios archivos.
- Crear y reutilizar un contexto de conexión.
- Ejecutar SELECT INTO.
- Declarar y recorrer iteradores nombrados.
- Convertir filas SQL en objetos Java.
- Insertar un cliente.
- Comprobar el número de filas afectadas.
- Actualizar con control del estado anterior.
- Eliminar cabecera y detalle en una transacción.
- Implementar INNER JOIN.
- Implementar LEFT JOIN tratando valores nulos.
- Consultar cuatro tablas relacionadas.
- Combinar resultados mediante UNION.
- Crear un informe con GROUP BY.
- Construir una transacción completa de pedido y líneas.
- Calcular importes mediante BigDecimal.
- Ejecutar COMMIT solamente al terminar la unidad de negocio.
- Ejecutar ROLLBACK ante errores.
- Conservar errores secundarios mediante addSuppressed.
- Examinar SQLState y código del fabricante.
- Cerrar iteradores, contextos y conexiones.
- Evitar que el repositorio cierre recursos que no le pertenecen.

- Identificar los puntos que dependen de Oracle, Db2 u otra implementación.

La cuarta entrega cerrará la guía con la comparación completa entre SQLJ y JDBC, la misma consulta implementada con ambas tecnologías, buenas prácticas, ejercicio final resuelto, ejercicios adicionales desde nivel básico hasta avanzado y el mapa conceptual definitivo.